

A Table of Data Is a Matrix

A dataset as one block of numbers, standardized in three ways, and why the times differ.

This unit, in one sentence

Computing with data: how data is stored, what an operation on it costs, and which processor carries it out.

- Today, data of three kinds becomes an array: a table, an image, a text. Each time: **one point**, then **a batch**.
- Three acts. Each has slides, then eight minutes in Colab, in one notebook for the whole meeting.
- Your group posts once in each act. One member posts, and every post begins with the group's number and the names of all its members.

Thirty datasets, one for each group

- Each group draws a number from 1 to 30. It names the group's dataset today, and its image and its word later.
- Every file is a table of numbers: 130,000 to 1,200,000 of them, from the UCI Machine Learning Repository.
- These slides use dataset 7.

Dataset 7: Beijing PM2.5

One row is one hour in Beijing, 2010 to 2014: the PM2.5 concentration of the air, the weather, and the date and hour as numbers.

Source: <https://doi.org/10.24432/C5JS49>

Read it with pandas

A **CSV file**: one record on each line, commas between the values, the names of the columns on the first line. `pd.read_csv` reads it into a **DataFrame**, a table in memory.

```
df = pd.read_csv(BASE + info["file"])
print("rows and columns:", df.shape)
df.head()
```

	year	month	day	hour	pm2.5	DEWP	TEMP	PRES	lws	ls	lr
0	2010	1	2	0	129.0	-16	-4.0	1020.0	1.79	0	0
1	2010	1	2	1	148.0	-15	-4.0	1020.0	2.68	0	0
2	2010	1	2	2	159.0	-11	-5.0	1021.0	3.57	0	0
3	2010	1	2	3	181.0	-7	-5.0	1022.0	5.36	1	0
4	2010	1	2	4	138.0	-7	-5.0	1022.0	6.25	2	0

From a table to a matrix

```
X = df.to_numpy(dtype=float)      # a NumPy array of decimal numbers
n, m = X.shape
rows = X.tolist()                # the same numbers as a list of n lists
```

```
n = 41,757 rows (points), m = 11 columns (dimensions), n x m = 459,327 numbers
type of rows: list of list
type of X:      ndarray with entries of type float64
first row, from the list: [2010.0, 1.0, 2.0, 0.0]
first row, from the array: [2.01e+03 1.00e+00 2.00e+00 0.00e+00]
```

- A **row** is one point with m coordinates. A **column** is one feature. The entry in row i and column j is x_{ij} .
- `rows[i][j]` in a list of lists, `X[i, j]` in a NumPy array: the same numbers, stored in two ways.

Standardization

Every column is brought to mean 0 and standard deviation 1, so that columns on different scales can be compared. For column j :

$$\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij} \quad \sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \mu_j)^2} \quad z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

- Divide by n , as NumPy's `std` does. (pandas divides by $n - 1$.)
- A program that follows the three formulas with loops visits every number **three times**: $3 \times n \times m$ visits.
- Dataset 7: $3 \times 459,327 \approx 1.4$ million visits. At 100 nanoseconds a visit, about 0.14 seconds. Keep that guess.

Version 1: loops on Python lists

```
def standardize_loops(rows):
    n = len(rows)
    m = len(rows[0])
    means = [0.0] * m
    for j in range(m):
        total = 0.0
        for i in range(n):
            total += rows[i][j]
        means[j] = total / n
    stds = [0.0] * m
    for j in range(m):
        total = 0.0
        for i in range(n):
            d = rows[i][j] - means[j]
            total += d * d
        stds[j] = math.sqrt(total / n)
    result = []
    for i in range(n):
        new_row = []
        for j in range(m):
            new_row.append((rows[i][j] - means[j]) / stds[j])
        result.append(new_row)
    return result
```

Written by an AI assistant to stated requirements: lists only, no NumPy, divide by n, return a new list and leave the input as it was.

Version 2: the same loops on a NumPy array

```
def standardize_array_loops(X):
    n, m = X.shape
    means = [0.0] * m
    for j in range(m):                # first pass: the mean of each column
        total = 0.0
        for i in range(n):
            total += X[i, j]
        means[j] = total / n
    sds = [0.0] * m
    for j in range(m):                # second pass: the standard deviation of each column
        total = 0.0
        for i in range(n):
            d = X[i, j] - means[j]
            total += d * d
        sds[j] = math.sqrt(total / n)
    Z = np.empty((n, m))              # a new array for the results
    for i in range(n):                # third pass: the new entries
        for j in range(m):
            Z[i, j] = (X[i, j] - means[j]) / sds[j]
    return Z
```

Two differences: `X[i, j]` in place of `rows[i][j]`, and a result array made with `np.empty((n, m))`. **Faster or slower than the lists? Write down your guess.**

Version 3: one operation on a whole array

```
v = np.array([1.0, 2.0, 3.0, 4.0])
print("v      =", v)
print("v + 10 =", v + 10)      # 10 is added to every entry
print("v * v  =", v * v)      # each entry is multiplied by itself
print("sum    =", v.sum())
print("mean   =", v.mean())
```

```
v      = [1.  2.  3.  4.]
v + 10 = [11. 12. 13. 14.]
v * v  = [ 1.  4.  9. 16.]
sum     = 10.0
mean    = 2.5
```

NumPy applies an operation to **every entry** of an array in one call. A computation written this way, with no Python loop over the entries, is **vectorized**. The loop still exists; it runs inside NumPy.

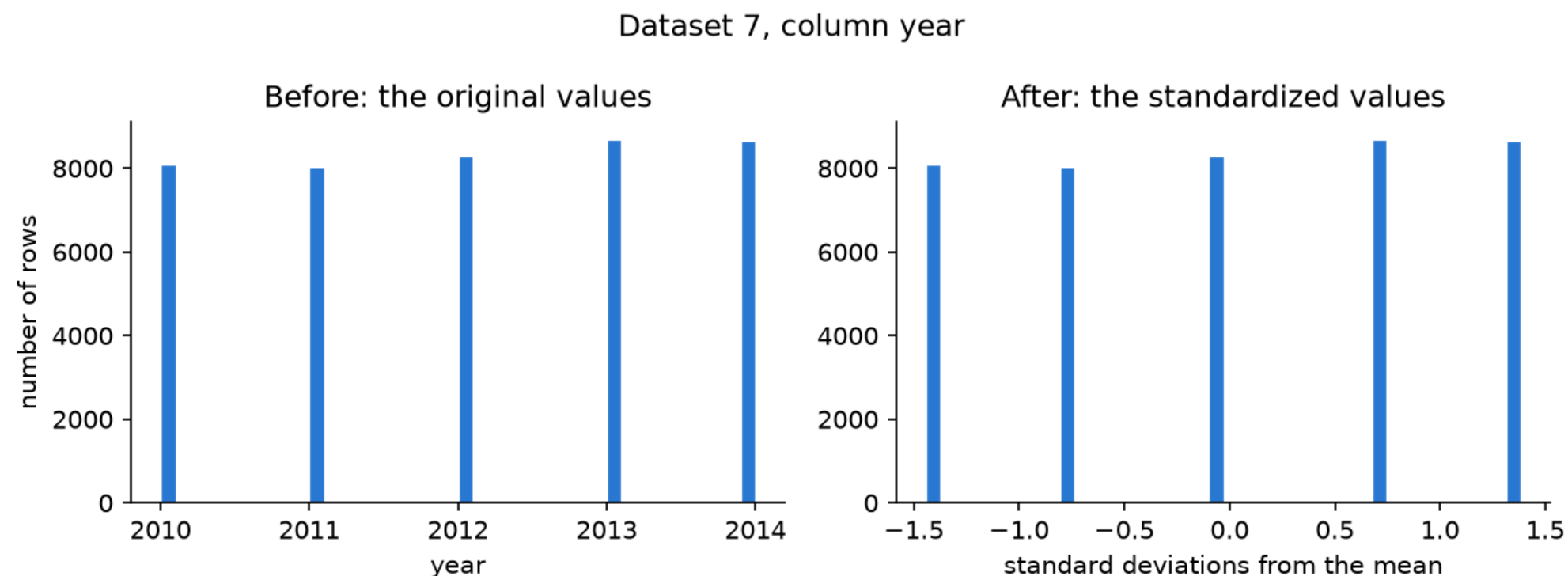
Standardization, vectorized, and broadcasting

```
def standardize_vectorized(X):  
    mu = X.mean(axis=0)           # the m column means  
    sigma = X.std(axis=0)         # the m column standard deviations  
    return (X - mu) / sigma       # broadcasting: n rows and m columns, combined with m numbers
```

```
shape of X:      (41757, 11)  
shape of mu:     (11,)   
shape of X - mu: (41757, 11)
```

- `X.mean(axis=0)` runs down the rows and leaves one number for each column: m numbers.
- `X - mu` subtracts m numbers from an array with n rows and m columns. NumPy subtracts `mu[j]` from every entry of column j , as if the m numbers were written again on every row. That rule is **broadcasting**.
- The division by `sigma` works the same way. The three loop passes of Versions 1 and 2 are what this one line replaces.

What standardization does to one column



	minimum	mean	maximum	std. deviation
before	2010	2012	2014	1.415
after	-1.443	-6.935e-14	1.383	1

Measure the three versions

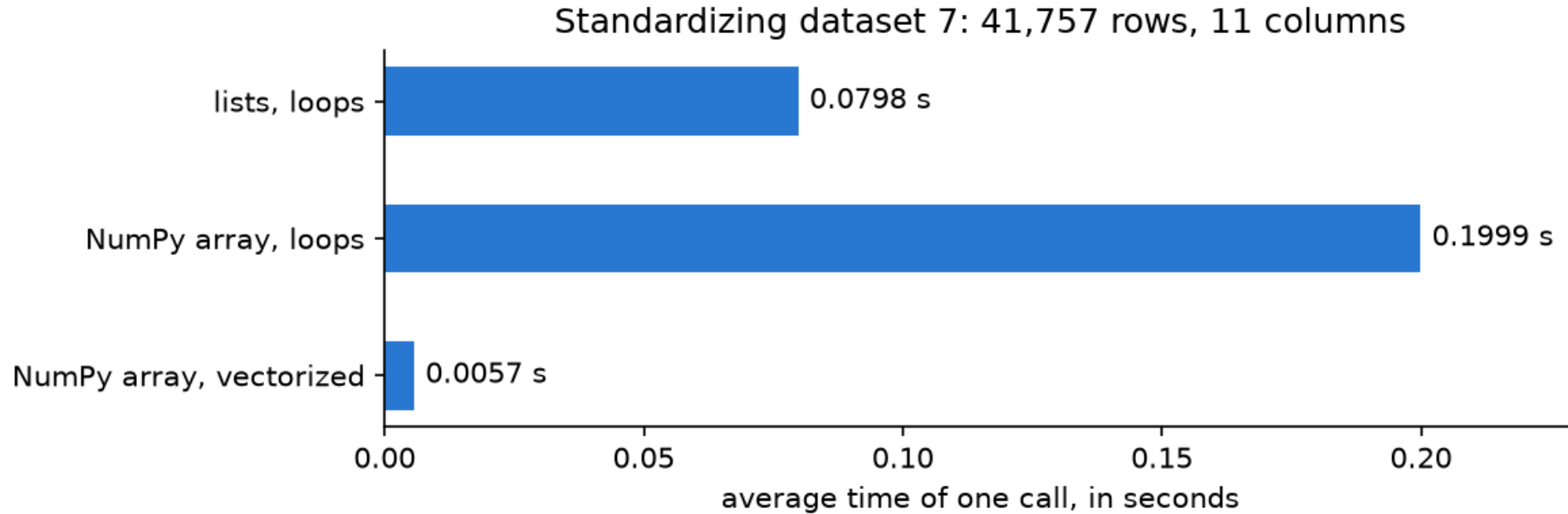
```
def average_time(function, data, repeats=5):  
    """The average time of one call function(data), in seconds, over several calls."""  
    total = 0.0  
    for _ in range(repeats):  
        start = time.perf_counter()           # the clock before the call  
        function(data)  
        total += time.perf_counter() - start   # the time that this call took  
    return total / repeats
```

Dataset 7: n = 41,757 rows, m = 11 columns, n x m = 459,327 numbers

version	seconds per call	nanoseconds per number	ratio to vectorized
lists, loops	0.0798	173.7	14.1
NumPy array, loops	0.1999	435.2	35.3
NumPy array, vectorized	0.0057	12.3	1.0

One measurement is not reliable: the machine does other work at the same time. Repeat, and average.

The three times



Dataset 7 (Beijing PM2.5), $n = 41,757$, $m = 11$. Nanoseconds per number: lists 173.7, array with loops 435.2, vectorized 12.3.

- The loops on the array are the **slowest**. The vectorized line is tens of times faster than the lists.
- Time per number: the time of one call divided by $n \times m$. It changes little with the size of the dataset, so groups can compare.

Why the times differ

- **What Python does for each pass of a loop.** The interpreter reads several bytecode steps, looks up each name, inspects the type of each value to decide what `-` and `/` mean, and creates a new object for each result: 24 bytes for a decimal number, 8 more for its place in a list.
- **What NumPy does for one operation.** The array is one block of 8-byte numbers of one type. `X - mu` is a single call into compiled code, whose loop does no lookup and builds no object, and one instruction can handle several numbers.
- **Why the loops on the array are the slowest.** The loop still runs in the interpreter, and each read of `X[i, j]` builds a new Python object for one number of the block.

An array is fast when the loop over its numbers runs inside NumPy.

Now in Colab: Act 1

- Open the **class notebook** from the meeting's page: one notebook for the whole meeting. **Save a copy in Drive**, type your group's number, run the setup cell.
- **Act 1:** run the cell that standardizes your dataset in the three ways. Read its table and its chart.
- Choose a column (`COLUMN = ...`) and look at its two histograms.
- **Post** on the discussion board: the printed line, the chart, the two histograms, and one sentence on what surprised you. One member posts; begin with your group's number and the names of all its members.